

Reflection and Approach

This is a Natural Language Processing (NLP) project. In this project, I built a system that can automatically read and understand news articles.

The NewsBot Intelligence System can:

-  Preprocess news articles using advanced text cleaning techniques
-  Classify articles into categories (Politics, Sports, Technology, Business, Entertainment, Health)
-  Analyze sentiment and emotional tone of news content
-  Identify patterns using TF-IDF, POS tagging, and syntax parsing
-  Generate insights for media monitoring and business intelligence

This project was built using:

- spaCy and NLTK: the main libraries for text processing and NLP tasks
- scikit-learn: used to train and evaluate the classification models
- BBC News dataset: a public dataset of 2,225 news articles, used to train and test the system
- Google Colab: the development environment used to write and run the code

This project follows a modular development approach. Instead of writing all the code in one large file, each function was built and tested separately in its own notebook. Once each function was tested and working correctly, it was exported into its own Python file inside the 'newsbot/' folder. Finally, all these functions were imported together into one main notebook, where the complete NewsBot system was built and tested with real news articles.

Preprocessing Module

First, I installed all the libraries needed for this project, such as spaCy, NLTK, and pandas. Then, I mounted Google Drive. This is where the dataset is stored, and where every notebook needed to build the NewsBot is saved. Working in Drive means the files stay saved even after the Colab session ends. After mounting Drive, I loaded the dataset and checked that it was imported correctly using `head()`, `shape`, and `info()`. This confirmed that the data was loaded properly before moving on. Next, I built the `clean_text()` function. These functions use regular expressions to remove things that add noise to the text, such as URLs, email addresses, and HTML tags. I tested this function on the first article in the dataset to check that it worked as expected. After that, I built the `preprocess_text()` function. This function uses spaCy to tokenize the text (split it into individual words) and lemmatize it (reduce each word to its base form). It also removes stop words and punctuation, since these do not add useful meaning for later analysis. I tested this function as well and confirmed that it returned a clean list of

lemmas. Finally, I exported both functions into a single file, `preprocessing.py`, using `%%writefile`. This file will later be imported into the main NewsBot notebook, where the complete system is built.

TF-IDF Module

I start the same way as before: mounting Google Drive, loading the dataset, and checking that it loaded correctly. This first step is repeated in almost every notebook in this project, since each one runs in its own separate Colab session. Next, I import the functions I already built and tested in the previous notebook (`clean_text` and `preprocess_text`) and apply them to the dataset to get it ready for analysis.

Once the text is clean and processed, I build the TF-IDF vectorizer. I set `max_features` to 5000, which is a conservative number, enough to capture the most useful words without making the dataset too large to handle. I also use `ngram_range=(1,2)`, meaning I look at both single words (unigrams) and pairs of words (bigrams). This helps me keep more information about how groups of words work together, without growing the vocabulary too much. For `min_df`, I use 2. This means I ignore any word that appears in only one document, since it could be a typo or a very rare word that doesn't help me understand the data. For `max_df`, I use 0.8, which means I ignore words that appear in more than 80% of the documents. These words are so common that they may act like "hidden" stop words, adding no useful information.

After that, I run `fit_transform` on the processed text and get the feature names. With these, I build the final TF-IDF DataFrame.

Finally, I build the `get_top_tfidf_terms()` function. This function shows the top words (in this case, the top 10) that are most important for a given category. It takes three arguments: the TF-IDF DataFrame, the category I want to look at, and how many top words I want to see. I tested the function to confirm it works correctly and then exported it into a `.py` file, so it can be imported later in the main NewsBot notebook.

POS Analysis Module

For this notebook, I only needed to import `clean_text`, since I do not want to remove stop words here. Removing stop words would take away important information about how adverbs and other grammar elements are used, which is exactly what I want to analyze in this module. For the `analyze_pos_patterns` function, I used `spaCy` and `Counter` to build a dictionary for each text, containing the proportion of each POS tag found in that text. I then tested the function by going through each row of the dataset, applying the function, and adding the category and article ID to the resulting dictionary. After that, I converted all the results into a DataFrame.

Finally, I exported the function into a `.py` file.

Syntax Parsing Module

I import `clean_text` again, since for this function I do not want to remove stop words, as I want to do semantic analysis, and stop words carry grammatical information I need.

To build the function, I use `spaCy` to process the text, then go through each token, skipping spaces and punctuation, and collect its dependency relation. I also separate out the subjects and objects of each sentence, and group full noun phrases, using `spaCy`'s built in noun chunk detection. At the end, I count how many times each dependency relation appears and return everything as a dictionary.

I tested the function with a very simple sentence, to check that it worked correctly, and, as I can see, the analysis is quite good. After that, I exported the function into a `.py` file.

Classifier Module

I start by loading the dataset and checking that it is loaded correctly. Then I import `train_test_split` to divide the dataset into a training set and a test set, using an 80/20 split. After that, I split the data into the usual structure (`x_train`, `y_train`, `x_test`, `y_test`), and check that the split worked correctly using `head()` and `shape()`.

Next, I encode `y_train` and `y_test` using `scikit-learn`'s `LabelEncoder`, which converts the category names into numbers the model can work with. I then import the functions built in the previous notebooks and apply them to `x_train` and `x_test`, to build the final dataset using both the TF-IDF analysis and the POS analysis.

After that, I ran cross-validation to choose the model that performed best. In this case, the best model was `LinearSVC`. I also ran a randomized search to try to improve it, but the results showed that the base model already performed better than the tuned versions. So, I trained the base model, and evaluated it, getting strong and consistent metrics, in line with what I saw during training. Finally, I saved the model using `joblib`, so it can later be used to build the NewsBot agent.

Sentiment Analysis Module

For this notebook, I use `NLTK` instead of `spaCy`, since `spaCy` does not have a built-in sentiment tool. I use `VADER`, which is `NLTK`'s sentiment analyzer, made specifically for this task. I did not apply `clean_text` here, since `VADER` works better with the original text. It uses things like capital letters and punctuation as signals of emotional intensity, and cleaning the text would remove that information.

For the `analyze_sentiment` function, I pass the raw text into `VADER`'s `polarity_scores()` method, which returns a dictionary with a compound score (an overall sentiment score from -1 to 1), and three more values: how positive, neutral, and negative the text is. Then, based on the compound score, I add a simple label: positive, negative, or neutral. I tested the function with three short example sentences, one clearly positive, one clearly negative, and one

neutral, just to confirm the function behaves as expected. After confirming it worked correctly, I exported it into a .py file.

NewsBot Agent Module

This is the final notebook of the project. Instead of building and testing one function, here I bring everything together into one working agent. First, I import all the functions built in the previous notebooks: `clean_text`, `preprocess_text`, `analyze_pos_patterns`, `extract_syntactic_features`, and `analyze_sentiment`. I also load the trained components saved earlier with `joblib`: the label encoder, the TF-IDF vectorizer, the classifier model, and the list of feature columns used during training. Then I build the `NewsBotAgent()` function. Given any raw article text, this function: 1. Cleans and preprocesses the text, and turns it into a TF-IDF vector 2. Extracts the POS pattern features for the same text 3. Combines both sets of features, and lines up the columns to match exactly what the model saw during training 4. Predicts the article's category 5. Analyzes the sentiment of the original (uncleaned) text 6. Extracts the syntactic features (subjects, objects, noun phrases) of the original text All of this is returned together as a single result.

Results on real-world articles

I tested the agent with a real, unseen article from the test set, and then with three new real-world articles, written about current events that the model had never seen before.

The first article, about Venezuela, talks about the recent earthquake that struck the country. The agent did not classify the category correctly, but it did correctly capture the sentiment, identifying the article as clearly negative, which matches the tone of a disaster story.

The second article, about Spain, talks about Spain's win at the World Cup. Here, the agent correctly identified both the category (sport) and the sentiment (positive), matching the fact that the article describes Spain winning the match.

The third article, about Disney, talks about Disney acquiring Marvel. The agent correctly classified this as a business article, since it is a story about a company acquisition. Overall, the agent showed solid performance, correctly identifying category and sentiment in most cases, with some room for improvement, particularly in distinguishing categories for articles that mix topics, such as disaster news that also involves numbers, locations, and statistics.

Personal Reflection

This project helped me get closer to technologies I had never used before, like `joblib`, and gave me a much deeper understanding of NLP in practice. Working through guided notebooks is very different from actually building your own pipeline from scratch. Here, I had to think through real problems with my data and with the NLP functions I was building, instead of just following steps that were already laid out for me. Building the project in a modular way, with each function developed and tested in its own notebook before being exported into the final

agent, taught me a lot about how real NLP systems are structured. I also ran into practical issues I hadn't considered before, like keeping the TF-IDF and train/test transformations consistent, avoiding data leakage, and making sure new, unseen text could be processed using the exact same feature structure the model was trained on. Testing the final agent on real, current news articles, not just on examples from the training data, was probably the most valuable part of this project. It showed me that a model can perform very well on paper, but still struggle with edge cases in the real world, which is an important lesson for any system I build in the future.